J MUKHESH

2303A51813

Tree boundary traversal

```
class Solution {
  ArrayList<Integer> boundaryTraversal(Node root) {
    // code here
    ArrayList<Integer> res=new ArrayList<>();
    if(root.left==null && root.right==null){
      res.add(root.data);
      return res;
    }
    res.add(root.data);
    left(root.left,res);
    leaf(root,res);
    right(root.right,res);
    return res;
  }
  void left(Node root,ArrayList<Integer> res){
    if(root==null) return ;
    if(root.left!=null) {
      res.add(root.data);
      left(root.left,res);
    }else if(root.right!=null){
      res.add(root.data);
      left(root.right,res);
    }
  }
  void leaf(Node root,ArrayList<Integer> res){
```

```
        if(root==null) return ;
        if(root.right==null && root.left==null){
            res.add(root.data);
        }
        leaf(root.left,res);
        leaf(root.right,res);
    }
    void right(Node root,ArrayList<Integer> res){
        if(root==null) return ;
        if(root.right!=null){
            right(root.right,res);
            res.add(root.data);
        }else if(root.left!=null){
            right(root.left,res);
            res.add(root.data);
        }
    }
}
```

Left view

```
class Solution {
    public ArrayList<Integer> leftView(Node root) {
        // code here
        ArrayList<Integer> l = new ArrayList<>();
        if(root == null)
            return l;
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        while(!q.isEmpty()){
```

```
        int n = q.size();
        for(int i = 0; i < n; i++){
            Node cur = q.poll();
            if(i == 0)
                l.add(cur.data);
            if(cur.left != null)
                q.add(cur.left);
            if(cur.right != null)
                q.add(cur.right);
        }
      }
      return l;
   }
}
```

Right view

```
class Solution {
   public List<Integer> rightSideView(TreeNode root) {
      List<Integer> ans=new ArrayList<>();
      if(root==null) return ans;
      Queue<TreeNode> q=new LinkedList<>();
      q.add(root);
      while(!q.isEmpty()){
         int n=q.size();
         int c=0;
         for(int i=0;i<n;i++){
            c++;
            TreeNode cur=q.poll();
            if(cur.left!=null) q.add(cur.left);
```

```
            if(cur.right!=null) q.add(cur.right);
            if(c==n) ans.add(cur.val);
          }
        }
        return ans;
    }
}
```

Path from root to given node

```
class Solution {
    public void paths(TreeNode root,List<String> result,String s){
        if(root == null)
            return ;
        s += String.valueOf(root.val);
        if(root.left == null && root.right == null){
            result.add(s);
            return ;
        }
        else
            s += "->";
        paths(root.left,result,s);
        paths(root.right,result,s);

    }

    public List<String> binaryTreePaths(TreeNode root) {
        List<String> result = new ArrayList<>();
        paths(root,result,"");
        return result;
```

```
    }


}
```